

Apuntes Clase 29/09/23

José Eduardo Gutiérrez Conejo - 2019073558

ALmacenamiento de datos en Elastic Search

Se puede encontrar que desde la primera version de ES que hay tipos de nodos, cuando se define un stting con master, lo que hace es forzar a la bases a tratar el nodo como dicho tipo, es decir, cada nodo tiene una tarea definida y no puede realizar los trabajos que deberia de realizar un nodo de otro tipos, se necesita al menos de un master y un data node para desarrollo, y para produccion se necesitan mínimo 3 master y 3 data nodes. Cuando se tiene un servidor de ES, lo que ocurre es que se mapea a una ubicacion geografica, y se tiene un punto de falla alto, ya que la ubicacion puede llegar a fallar y no se puede recuperar la información. Charts, se puede encontrar en bases de datos no SQL, son porcines de un data set que uno maneje, si se tiene un dataset, y se quiere tener una cantidad de 5 charts, al tener 5 en un solo servidor, no genera sentido, ya que los charts quedan en el mismo servidor, y no se gana nada en un punto de vista de alta disponibilidad. Aunque se haya particionado, no se gana nada. Se tiene charts que son primary y otros que son replica. Si estos viven en el mismo servidor no se gana nada, la idea es que quedan distribuidas geográficamente. Cuando se pasa al mundo de produccion, se debe de hacer que la base de datos se configure con los mínimos mencionados, y cuando se definen los datos, se debe de tener un dataset que ser charteado, y debemos de tener minimo 2 replicas de esos datos. Normalmente, ES, no va a dejar que los charts primarios esten en el mismo servidor, ademas este es el unico que puede recibir actualizaciones, y lo que hace es replicarlas a los demás charts, si se dejan todos los primarios en el mismo servidor, lo que puede pasar es que la capacidad del cluster no se utiliza correctamente, y lo que pasaria es que un servidor realiza la mayor cantidad de trabajo, y los demas muy poco. Cuando un servidor se muere y tiene un replica chart, cuando es primario, esto lo que pasa es que se necesitan al menos 3 y se procede a realizar un chart election, alguno de los 2 nodos vivos se convierte en el maestro y promueve el chart, y lo que ocurre es que inicia un votacion (hay factores que pueden influir en esto, como el timestamp), y alguien va a asumir mayoría, y se va a promover una de las replicas de las que estaban como un chart replica, se va a pasar a chart primario y la base de datos se recupera, un indice que tiene charts se tiene un status, green: todo bien, yellow: no se tiene todos los necesarios; puede que haya alguna replica caída, red: cuando se cae un servidor y no se tenia replicas para solventar el problema. Si el nodo 3 se agrega mas temprano que uno, se puede elegir la eleccion del chart para que elija el mas reciente. Tambien puede ser afectado por la geolocalizacion (el más cercano). Tambien pueden afectar la cantidad de particiones que tiene los servidores. Se puede agregar un nuevo servidor y ES se va a encargar de distribuir los trabajos, chart allocation, para comodar los charts. Esto es cuando se habla de que se tiene todos los roles dentro de un mismo servidor, master, data, ingest, y transform,, esto no es conveniente. Una base de datos que dura 60 en realizar un

insert no es nada buena, pero durante este periodo se consume cpu y memoria y afecta el rendimiento del servidor entero. Cuando se da un momento de altas lecturas o altas escrituras, hay nodos que trabajan mucho dando estas consultas, y si se mantiene todos los roles en un mismo servidor, y hay una conexión entre nodos, ES es el 9300, esta conexión es manejada por certificados, se llama el transport layers, ayuda a tener el health de las bases de datos, todos los nodos preguntan por la salud de una cierta replica, y si se tiene ocupada el sistema, esta consulta puede llegar a durar mucho, lo que puede causar que el sistema cree que el nodo está caído, lo que inicia el proceso de elección, y esto puede llegar a generar bajo rendimiento en la base de datos. Siempre la prioridad es tener disponible los datos para los usuarios, se tiene tres master como mínimo, y se va a tener 3 data nodes, y en algunos casos, dependiendo del workload, se pueden manejar 3 ingests, El ingest se transforma en una pasarela si no se manejan transformaciones, y luego lo manda al servidor, y esto es una pérdida de hardware, ya que son recursos que no fueron utilizados. Se pueden correr scripts en los pipelines de inserción, y se pueden llamar, APIs, estandarizar, y transformar la información, entonces el crear nodos de ingestión dedicados, es una buena idea. Se le puede poner un tipo de load balancer a los ingest y a los data nodes, el roundrobin manda peticiones en ciclo a los diferentes nodos, una a la primera, la siguiente al segundo y así sucesivamente. Cuando un servidor no tiene donde guardar la información, se comunica con el nodo que si los tiene, estas consultas se pueden realizar al master node. No es común tener los 3 servidores en una misma localización geográfica, nosotros siempre debemos de tener los juegos de servidores en diferentes localizaciones, ya que si el sistema se cae, tener diferentes replicas en diferentes zonas, caso donde no tenga sentido tener replicas de los charts Las escrituras siempre son sincrónicas, cuando se hacen en el primary, una vez se realiza la escritura, el nodo con el primary es el encargado de mandar la información a las replicas y asegurarse que los datos se inserten de manera correcta y brindar esta información a los master nodes ya que son los encargados de dar este tipo de datos, cuando se tiene heavy writes, se quiebra este tipo de esquema y se quitan las replicas y se deja un solo chart primario sin charts secundarios para que las escrituras sean más rápidas, se reducen el trabajo del servidor o donde se tiene el chart primary pero se manejan mucho más rápido los procesos de datos, esto no es muy seguro pero es mucho más eficiente. En el momento en que entra un write, entra la info y se escribe las replicas, mientras que cuando se tiene un read, si se tienen 3 usuarios que hacen lecturas a la base, los 3 usuarios, uno lee, los otros dos esperan y así sucesivamente, al tener solo una replica o un chart primario, esto se puede convertir en un cuello de botella, en servicios con muchos usuarios esto es un problema y en este caso, sirve tener un montón de replicas, muchas más replicas se tengan las lecturas son más eficientes, si se tiene un sistema que tenga muy altas escrituras, lo mejor es siempre tener un servidor con un chart primary, si no importa el acceso rápido, ya que no importa si se pierde la información de una hora, se pueden realizar backups de una hora de los que entra en la base de datos, esta es la única forma de mantener ciertos datos seguros, (pérdida que pueden ser de una hora max) Se pueden desactivar los

índices en la base de datos, si hay un momento de altas lecturas, se apaga el índice y se relizan los appends y entonces una vez pase el momento de alta concurrencia se enciende el índice y entonces se actualiza para ser utilizado, altas escrituras y bajas lecturas, un solo primary, cuando es muchas lecturas más replicas, y cuando es ambas, se mete hardware. Si se tiene un índice pequeño, de 10 gb, hacer los charts en este caso no tiene sentido, se gasta más tiempo en coordinación, health checks y recuperar información, entonces cuando es pequeño, recibe pocas escrituras, y muchas lecturas, entonces en este índice es importante crear replicas del índice y así mejorar el rendimiento. Cuando se tiene un dataset que pesa un giga, se pueden realizar charts de 250 megas, los masters se dan cuenta y se mete la info al cluster, se crea el índice y se configura con las particiones, el master de los masters se puede distribuir los datos de los charts y lo da de forma equitativa, y se empieza a preguntar como están los charts, y cuando no se responde, los masters cambian el color del índice, y van a decidir en quien se convierte en el nuevo nodo primary del chart que murió, para poder realizar esto, se especifica el número de charts y el número de replicas, entonces se distribuyen las replicas (de los charts), Un nodo propone y los demás aceptan si es correcto realizar esto. Los master nodes solo preguntan como está todo y distribuir los datos, y dan la organización donde viven los charts, cuales data nodes si se detecta un fallo del chart primario, se maneja de la siguiente forma, no se pueden recibir actualizaciones ni escrituras y se define quien es el nuevo chart primario, cuando se convierte en primario aceptan los updates, cuando se cambia el primary, se debe de crear una nueva replica para mantener el mínimo de replicas especificado. Es difícil darse de patrones que no tengan master, ni ingest ni data nodes, pero si se da. Se tiene los masters y en vez de data nodes, se tiene data content nodes, esto en producción, lo que sucede es que en ES se tiene algo que se llama alias de index, que dice que se define un index, la app ingresa al índice y este índice se empieza a agrandar y cuando más info, más grande, y se puede realizar algo distinto, alias que se llame documents, el cual tiene un índice, y se define con el nombre del alias y se decide que info debe de tener, mes día año, y en el alias se define el tiempo que va a vivir un alias y esto implica que cuando la aplicación está escribiendo, ej en documents y empieza a escribir en un alias, y cuando se termina el índice de esta fecha, el alias y el índice van a quedar abriendo y cerrando, esto para que el índice no se haga enorme, si el índice no es grande, las búsquedas no son tan profundas, y son más rápidas, y se tiene la garantía de que los índices son diferentes y se puede reducir el tamaño de los archivos y se convierten a más manejables, y la probabilidad de que la info esté en un mismo archivo es bastante alta, después se tiene el concepto de layers, data content, data hot, data warm, data cold y data frozen. En el momento que el índice está activo, se maneja en data content, los charts van a estar guardados en data content, al terminar el día se mueve el índice a data hot y el más nuevo pasa a data content, los datos van de layer en layer, cada vez a uno más frío. El índice nuevo se va a tener el hardware más caro y más eficiente. Luego los hot es un poco menos potente, luego warm es más bajo, el frío incluso más, hardware de instancias baratas contra un uso de un object storage, S3, BCS azure blocks

y el frozen es directamente el object storage, es muy barato. Cuando se tiene un data set medio o pequeño y se pueden levantar muchos registros a memoria, se pueden crear máquinas que son optimizadas para esto, dependiendo de las necesidades de lo que se va a relajar se pueden elegir máquinas que cumplen con estos requisitos. Si se tiene un tera de almacenamiento para tres instancias en data content, sale el 1700 dólares al mes, pero si se tiene esa misma cantidad, si los datos no tienen tantos accesos al día, no tiene sentido utilizar el mismo hardware en los diferentes layers, y esto gasta mucha plata, después entonces se pasa los data content la máquina más cara y van bajando en precio conforme baja el tipo de datos. se debe conocer la estructura y como se van a manejar estos, se hacen loadtest con gatlin y se empiezan a determinar cuales son las máquinas y la topología de la base de datos, de una instancia otra casi cae 700 dólares, y la reducción de precios es muy alta, al rededor de un 60m 70 por ciento de reducción, nuestra mayoría de usuarios están felices y la gente que no lee mucho están en hardware menor y es más lento, en un servicio S3 el precio de mantener 100 tb al mes, el precio es de 2 300 dólares, y en servicios que son los discos que corren las bases de datos, el precio es de 15 000 dólares, por esto se debe de clasificar los datos, y sacar info que no se debe leer de las mejores máquinas, los datos no siempre se leen. Por ejemplo con glacier, que son datos que solo se leen una vez cada 6 meses, baja a como 100 dólares, que es una reducción muy importante en esto, de 1500 dólares a 100 dependiendo de la máquina, por esto es importante mover los datos en los diferentes tiers para reducir costos, en google se encuentran bases sql que hacen algo parecido pero son difíciles de manejar, cuando se realizan estimaciones para ver el tamaño de la base de datos, es un poco difícil estimar esto, pero uno utiliza una clase de esquema que permite dar a conocer como realizar esta estimación, el precio final no va a ser exactamente lo que se estima, y cuando se gradúa se puede llegar a tener que estimar una base de datos, y siempre se debe de meter un pequeño colchón para estimar hacia arriba, entre gente técnica es fácil saber que es una estimación pero la única forma de validar estas pruebas es normal utilizar herramientas como gatlin y prometheus y cuando se realizan las pruebas, se agarran los datos y se protegen con el dataset completo, hay guías de sí que se utilizan para estimar la cantidad de documentos que se van a mantener y se van a estimar los crecimientos de la base de datos, por ejemplo si se tiene mucha escritura, y lo que nos ayuda a estimar es el tamaño que va a tener el dataset en disco, el concepto de raw dataset, se puede estimar con un pedazo del dataset, cuantas copias se deben de tener, en cuantas localizaciones se deben distribuir. esto es importante ya que se tiene un disco de 400 gb y se tiene en memoria principal 8 gb, cuando una app realiza lecturas, lo que pasa es que la app va a tener que hacer paginación hacia el disco, y la primera vez que se llama, debe de ir a traer info al disco y en ES, normalmente un ratio de 1 a 30 es lo que se llama hot, e implica que garantiza una velocidad alta. cuando se estima, se debe de pensar en esto, si se tiene un sistema que mantiene una base de datos y hay un tiempo donde no se lee mucho, no es bueno tener un hot reads, el tamaño average del documento me ayuda a predecir la cantidad total de documentos que se van a manejar en un tiempo aproximado, 6 meses, 1 años, etc, cuando se tiene eso

se define cuanto tamaño debe de tener, lo siguiente es calcular el dataset raw y siempre se pone un extra (colchon), 10 % de error, se puede sacar entonces la memoria a utilizar. entonces se agarra y se busca un amaquina que se asemeje a los datos obtenidos y se relizan pruebas con prometheus y gatling y una vez se tiene todos y ya una vez sse realiza todo se puede asegurar que se realizo un buena estimación.